

Stockfish Integration Plan

SD2 Project Group-15 — Iteration 3 — Chess Bot Opponent

1. Overview

For iteration 3 we will integrate the Stockfish chess engine to provide a computer opponent. Stockfish is an open-source UCI (Universal Chess Interface) engine: our Java application will launch the Stockfish executable as an external process and communicate with it by writing UCI commands to its stdin and reading the engine's responses from its stdout. The engine runs on a separate thread so the Swing UI stays responsive while the bot is thinking.

2. Goals & Non-Goals

In scope:

- Launch Stockfish as a subprocess and communicate via the UCI protocol.
- Offer a “Play vs Bot” mode where one side is driven by the engine.
- Convert our internal board state into FEN / move-list so Stockfish can read it.
- Parse *bestmove* output, convert it back to `IndexPosition`, and play it on the board through **GameService.handleSquareClick**.
- Expose a difficulty setting (skill level and/or search depth / movetime).
- Handle engine lifecycle cleanly: start on new game, stop on window close.

Out of scope for this iteration:

- Opening book, endgame tablebases, multi-PV analysis view.
- Online multiplayer; bundling a full GUI analysis board.
- Shipping the Stockfish binary itself — the user installs it locally.

3. Architecture

We will add a new **Services/BotService** class that owns the engine subprocess, plus a small **FenService** (or static helper) that converts our `Piece[][]` board into FEN. **GameService** stays the single authority for moves; the bot only calls the same `handleSquareClick` entry-point the user does. This keeps rules (check, castling, en passant, promotion) in one place.

Component	Responsibility	Status
Game / Board	UI + user input (Swing). Unchanged.	existing
GameService	Rules, move execution, turn tracking.	existing
CheckService / DrawService	Check, checkmate, stalemate, draws.	existing
BotService	Owns Stockfish process; sends UCI, reads bestmove; asynchronously returns the chosen move.	NEW
FenService	Converts current <code>Piece[][]</code> + side-to-move + castling rights + en passant target into a FEN string.	NEW
UciMoveParser	Converts bestmove strings like e2e4 / e7e8q into <code>IndexPosition</code> (NEW promotion piece).	NEW

BotController (in Game)	listens for end-of-turn, asks BotService for a move, applies it	in the EDT.
-------------------------	---	-------------

Data flow (bot's turn): user makes move → handleSquareClick updates board → turn flips to bot color → BotController calls BotService.requestMove(fen, depth) on a worker thread → Stockfish replies bestmove e7e5 → parser produces IndexPosition(4,4) → SwingUtilities.invokeLater replays that move through the same handleSquareClick path used by the human.

4. UCI Protocol — What We Actually Need

UCI is line-based text. We only need a small subset:

- uci — handshake; engine replies with id/options, then uciok.
- isready — engine replies readyok; used as a sync point.
- ucinevgame — reset engine state between games.
- setoption name Skill Level value N — difficulty (0–20).
- position fen <FEN> or position startpos moves e2e4
- go depth 10 or go movetime 1000 — start searching.
- bestmove <move> — engine's reply we care about.
- quit — ask engine to exit cleanly.

Example session we will send/receive:

```
> uci
< id name Stockfish 16 ... uciok
> setoption name Skill Level value 5
> ucinevgame
> isready
< readyok
> position fen rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
> go movetime 1000
< bestmove e7e5
```

5. BotService — Implementation Sketch

Core skeleton (details will live in src/Services/BotService.java):

```
public class BotService {
    private Process engine;
    private BufferedReader in;
    private BufferedWriter out;

    public void start(String exePath) throws IOException { ... }
    public void setSkillLevel(int level) { send("setoption name Skill Level value " +
level); }
    public String requestBestMove(String fen, int movetimeMs) { ... }
    public void newGame() { send("ucinevgame"); waitReady(); }
    public void shutdown() { send("quit"); engine.destroy(); }

    private void send(String cmd) { out.write(cmd + "\n"); out.flush(); }
    private void waitReady() { send("isready"); readUntil("readyok"); }
}
```

Important: all blocking I/O with Stockfish must run on a worker thread (e.g. SwingWorker or a single-thread ExecutorService). Doing it on the Swing Event Dispatch Thread will freeze the window while the engine searches.

6. FEN Conversion — Mapping Our Board to Stockfish

Stockfish does not know about our Piece classes, so we build a FEN string. A FEN has six fields: piece placement, side to move, castling rights, en passant target, halfmove clock, fullmove number.

Mapping rules:

- Rows: FEN starts from rank 8 (row index 7 in our array) and ends at rank 1. Iterate row = 7 → 0.
- Columns: left-to-right is file a→h, i.e. col 0→7.
- Pieces: King→K, Queen→Q, Rook→R, Bishop→B, Knight→N, Pawn→P. Uppercase for WHITE, lowercase for BLACK.
- Side to move: read `GameService.getCurrentColorToMove()` → “w” or “b”.
- Castling rights: derive from `canCastleKingSide/canCastleQueenSide`, or walk `moveStorage` to see if king/rook has moved; output KQkq as appropriate, or “-”.
- En passant target: if `getLastDoubleStepPawn()` is non-null, compute the square behind it (e.g. “e3”); else “-”.
- Halfmove clock: `DrawService` already tracks 50-move rule — reuse that counter.
- Fullmove number: $(\text{moveCounter} + 1) / 2$.

7. Parsing “bestmove”

Stockfish returns moves in long algebraic notation, e.g. e2e4, e1g1 (castling), e7e8q (promotion). The converter:

- Characters 0–1 → from-square; characters 2–3 → to-square.
- File letter → column (a=0 ... h=7); rank digit → row ('1'=0 ... '8'=7).
- Optional 5th char = promotion piece (q, r, b, n). Stash it; `GameService.executePromotion` currently auto-queens, so we will extend it to accept the chosen piece.
- Feed the resulting `IndexPosition` into `handleSquareClick` **twice**: first on the from-square (selects the piece), then on the to-square (performs the move) — exactly like a human click sequence.

8. UI / UX Changes

- New menu / dialog at startup: “Human vs Human” vs “Human vs Bot”, plus a choice of which color the human plays.
- Difficulty slider (Skill Level 0–20, or movetime 100–3000 ms).
- “Bot is thinking...” label or disabled board while Stockfish searches (prevents the user from clicking during the bot’s turn).
- Path-to-engine field (with a sane default) — the user tells us where stockfish is installed; store it in a small config file.
- Graceful error dialog if the executable is missing or the handshake fails.

9. Testing Strategy

- **Unit:** `FenServiceTest` — build several known board states (starting position, after 1.e4, mid-game with castling rights lost, en passant available) and assert the exact FEN string.
- **Unit:** `UciMoveParserTest` — parse e2e4, e1g1, e7e8q, a7a8n → verify from/to indices and promotion piece.
- **Unit (mocked engine):** `BotServiceTest` — inject a fake `BufferedReader/BufferedWriter` pair scripted with canned UCI responses so we can assert protocol correctness without launching Stockfish.
- **Integration:** a single slow test that actually starts Stockfish, plays the opening move from startpos, and asserts a legal response. Guarded by `Assume.assumeTrue(new File(path).canExecute())` so CI doesn’t fail for teammates who haven’t installed the engine.
- **Manual:** full game against the bot at 3 skill levels; verify castling, en passant, promotion, checkmate detection, resign/new-game.

10. Work Breakdown & Ownership

#	Task	Deliverable	Est.
1	Install Stockfish on each dev machine; Read the section README	Read the section README File format.	0.5 d
2	Implement <code>FenService</code> + unit tests.	<code>FenService.java</code> , <code>FenServiceTest.java</code>	1 d

3	Implement UciMoveParser + unit tests. UciMoveParser.java, UciMoveParserTest.java	0.5 d
4	Implement BotService (process I/O, UCI protocol, java requests & responses).	1.5 d
5	Wire BotController into Game: worker thread, EDT calls, java Swing & sqldo; state.	
6	Extend executePromotion to accept piece type (needs java update & tests engine).	0.5 d
7	Add new-game dialog: mode + color + difficulty. Swing dialog class	1 d
8	Integration test + manual test pass at 3 difficulty levels	0.5 d
9	Write iteration-3 report section describing PDF integration	0.5 d

11. Risks & Mitigations

Risk	Mitigation
Stockfish binary path differs per OS / machine	Config file + env var fallback; clear error dialog if not found.
Swing UI freezes while engine searches.	All engine I/O on a worker thread; update board via invokeLater.
FEN generator disagrees with engine & sqldo; tests against known positions	Units tests against known positions; cross-check with an online FEN tool during development
Promotion mismatch — engine picks wrong piece type	Extend executePromotion to accept a piece type; default stays queen.
Engine process leaks if window is force-closed	Register shutdown hook / windowClosing listener that calls BotService.shutdown
Castling / en passant info missing from FEN	Dedicated unit tests covering both flags; use DrawService's halfmove

12. Definition of Done

- Human can start a new game, choose color + difficulty, and play a full game against Stockfish to checkmate, stalemate or draw.
- All new classes have JUnit5 tests; existing test suite still passes.
- Closing the window terminates the Stockfish process (verified via Activity Monitor).
- README documents how to install Stockfish and where to point the engine path.
- Iteration-3 report includes this integration and the final class diagram.